🧩 Section 3: Running and Managing Containers

🎯 Section Goal

By the end of this section, you should be able to say:

> "I can run containers, manage their lifecycle,
> and understand how to configure stateful services like databases."

🧠 Why This Section Matters

Running a container once is nice.
Running it correctly is what matters in real life.

This section answers:

- How do I run real applications like databases?
- How do I connect to them?
- How do I manage environment variables and credentials?
- How do I stop, restart, and clean up?

🧪 Hands-On First: Running a Real Container

🎯 Lab Objective

Run a PostgreSQL database inside a container and connect to it.

👣 Step 1: Run a PostgreSQL Container (Without Port Mapping)

Run the following command:

```shell
docker run postgres
```

What happens:

- The container starts

- PostgreSQL is running inside
- But the database is not accessible from your machine yet

This is intentional.

🧠 Why Can't We Connect?

Key idea:

- Containers have their own network
- Ports inside the container are NOT exposed by default
- Even though PostgreSQL is running, it's isolated

An application can be running but still unreachable.

🌐 Port Mapping (Critical Concept)

Port mapping connects:

- A port on your machine
- To a port inside the container

This is how traffic flows in and out.

👣 Step 2: Run With Port Mapping and Configuration

Stop the previous container (Ctrl + C), then run:

```shell
docker run -p 5432:5432 -e POSTGRES_PASSWORD=mypassword postgres
```

Explanation:

- `-p 5432:5432` → Maps port 5432 on your machine to port 5432 in the container
- `-e POSTGRES_PASSWORD=mypassword` → Sets the root password environment variable
- `postgres` → The image name

Key Point:
Unlike stateless applications, databases need credentials.
Environment variables are how we pass configuration.

👣 Step 3: Connect to the Database

Open a new terminal and connect using psql:

```shell
psql -h localhost -U postgres -W
```

Alternative way to connect to the database

```shell
docker exec -it <container_id_or_name> psql -U postgres
```

When prompted, enter the password: `mypassword`

You should see the PostgreSQL prompt:

```
postgres=#
```

This confirms:

- The container is running
- Networking is working
- Port mapping is successful
- Credentials are correct

👣 Step 4: Test the Connection

Run a simple query:

```sql
SELECT version();
```

You should see the PostgreSQL version information.

This proves the database is fully functional inside the container.

🧠 Understanding What's Running

View Running Containers

```shell
docker ps
```

This shows:

- Container ID
- Image name
- Status
- Port mappings
- Environment variables being used

View All Containers (Including Stopped)

```shell
docker ps -a
```

Stopped containers still exist until removed.

🧠 Stateful vs Stateless Containers (Important Idea)

Nginx (Stateless):

- Serves web pages
- Doesn't care about previous requests
- Can be restarted without consequence

PostgreSQL (Stateful):

- Stores data
- Data persists across restarts (if volume is mapped)
- Needs credentials and configuration
- More complex lifecycle

This section teaches you about stateful containers.

⏹️ Stopping and Removing Containers

Stop a Running Container

```shell
docker stop <container_id>
```

Data in memory is flushed, but the container still exists.

Remove a Container

```shell
docker rm <container_id>
```

The container is deleted, but the image remains.

⚠️ Important: Data Loss in Containers

When you remove a container running PostgreSQL:

- **All data is lost** (unless using volumes)
- This is a key limitation of container lifecycle

Later sections will cover volumes for data persistence.

🧠 Containers Are Ephemeral (Important Idea)

Key mindset shift:

- Containers are not pets (persistent, long-lived)
- Containers are cattle (replaceable, disposable)

If something breaks:

- Stop it
- Remove it
- Run it again

This is normal and expected.

🧪 Mini Exercise (Student Activity)

1. Run PostgreSQL with a custom password
2. Connect to the database using psql
3. Create a simple table:

```sql
CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR(100));
```

4. Insert data:

```sql
INSERT INTO users (name) VALUES ('Alice');
```

5. Query the data:

```sql
SELECT * FROM users;
```

6. Exit psql and stop the container
7. Remove the container
8. Run a new container and verify the data is gone

Goal:

- Build confidence with the container lifecycle
- Understand that containers are temporary
- See why volumes are needed for persistent data

⚠️ Common Beginner Confusions

- "The database is running but I can't connect" → Port not mapped or credentials wrong
- "Where's my data?" → Removed the container (data is ephemeral)
- "Why can't I use the same password in multiple containers?" → You can, it's just configuration
- "What's POSTGRES_PASSWORD?" → Environment variable for database credentials

All of these are normal at this stage.

🧠 Section Takeaway

You now know how to:

- Run containers with configuration (environment variables)
- Expose applications for external access
- Connect to services running in containers
- Understand stateful vs stateless containers
- Stop and remove containers safely

You understand that containers are ephemeral and why that matters.

🚪 Conclusion

> "Running containers from images is powerful.
> But we need to solve one critical problem: data persistence.